SensAI — Work Plan in Layers

Four members · eight layers · four tasks per layer · ordered most-blocking first

---

Four members, eight layers, four tasks per layer. Each layer is a wave: the four tasks inside it are independent of each other, so all four people work at once. A layer's outputs are what the next layer stands on, so layers run in order. Layers 1–4 are blocking. Layers 5–6 are where the project earns its marks. Layer 7 is optional. Layer 8 is delivery. Ordering follows the catalogue's own star ratings: the six three-star features this plan commits to — T1, R2, A1, A2, EV1, EV2 — all land by layer 6, and nothing rated one star is allowed to displace them.

The four members

The columns never change, so each person follows one thread through the whole project.

Member	Owns
A · Core	Ollama transport, event stream, agent executor, orchestrator, router, config
B · Memory	SQLite, profile, facts, SRS, context builder, budgeting, compression
C · Knowledge	Tool registry, own tools, MCP client, RAG pipeline, the corrector spec
D · Trust & surface	Logging, metrics, trace view, judge spec, guardrails, CLI and Web UI

Four standing rules

1. **Do not start a layer until the previous layer's contracts are merged.** Not "finished" — *merged*. Three people building against a fourth person's unmerged branch is how this project fails.
2. **Two user stories per feature, written when the feature is specced, not at the end.** Each acceptance criterion must name something observable in the demo or in a test. This is a hard requirement in the brief.
3. **Every layer ends in something you can show.** If a layer produces no demo, the layer was mis-scoped.
4. **`num_ctx` and `model` are always explicit in the spec files.** Never inherit Ollama's defaults, or the same code behaves differently on each of your four laptops and no measurement is comparable.

The catalogue's ratings

Every task below carries the rating of the feature it delivers. B1 has a thumbs-up rather than stars, matching the brief's line that the base loop earns nothing on its own.

Rating	Features
★★★	T1 MCP · T2 Sandboxed exec · R2 Advanced RAG · A1 ReAct · **A2 Multi-agent** · EV1 Eval · EV2 Guardrails
★★	M2 Budgeting · M3 Memory · M4 Artifact doc · T3 Web search · R1 Basic RAG · A6 Cache · EV3 Logging · EV4 Adversarial · X2 Branching
★	M1 Persistence · T4 File access · T5 Structured output · A3 HITL · A4 Prompt versioning · A5 Persona · A7 Prompt opt · X1 Web UI · X3 Export · X4 Forms · X5 Scheduling
👍	B1 Functional CLI chatbot — mandatory, unrated

Committed: six of the seven three-star features. T2 is declined on purpose — a Japanese tutor has no honest use for running generated code, and the catalogue is explicit that a forced fit scores worse than a clean exclusion.

LAYER 1 · Foundations — the four frozen contracts (no feature ships yet)

Nothing else can be written until these exist. This is the only layer where the tasks are deliberately small and the interfaces matter more than the implementation.

- **A · The contracts** *(serves A2 ★★★)*. `core/events.py` (the event dataclasses), the `AgentSpec` type, the `Handoff` envelope, and the layered config loader (defaults < config.toml < env < CLI args). Stub implementations are fine; the *shapes* are what get frozen.
- **B · Storage** *(serves M1 ★, M3 ★★)*. SQLite schema and migrations, `journal_mode=WAL`, `foreign_keys=ON`, plus the async access pattern (single writer task or executor — decide now). Must include `messages.parent_id` and `agent_runs.parent_run_id` even though nothing uses them yet.
- **C · Ollama transport** *(serves B1 👍)*. `POST /api/chat` with NDJSON streaming, `/api/embed`, `/api/tags`, `/api/show`. Capture the final chunk's telemetry. Full error taxonomy: 404 model missing, connection refused, 400, and the mid-stream `error` line that arrives while HTTP is still 200.
- **D · Surfaces over the stream** *(serves B1 👍, EV3 ★★)*. A CLI renderer and a structured logger, both written against A's stubbed event types. Two consumers from day one is what proves the event stream is really decoupled.

> **Done when:** a throwaway script streams tokens from `qwen3.5:4b` to the terminal, a row lands in `turns`, and `pytest` passes on the error cases.

LAYER 2 · The base loop — B1 👍, M1 ★

- **A · The turn (B1 👍).** `run_turn()` as an async generator: guardrail stub → context stub → Ollama → events. One agent, no routing yet, but the shape is already multi-agent.
- **B · Session persistence (M1 ★).** Save and reload sessions, walk the message tree from `active_leaf_id`, and auto-inject the learner profile at init.
- **C · The model bake-off** *(unrated — it decides every spec's model)*. Twenty hand-written Japanese cases against `qwen3.5:4b`, `qwen3.5:9b` and `gemma4:12b-it-qat`. Record accuracy *and* tokens/sec at your real `num_ctx`. **This decides the `model` line in all five agent specs, so it cannot slip.**

- **D · Failure and feel (B1 👍).** Harden all four required error paths, add the CLI's slash commands, and make the streaming render actually pleasant (no flicker, clean Ctrl-C).

> **Done when:** you can quit, restart, and the assistant remembers who you are — and you have a table saying which model you chose and why.

LAYER 3 · Memory that is real — M3 ★★, T5 ★, T1 ★★★, EV3 ★★

- **A · Structured output (T5 ★).** JSON schema on the chat call, validation, exactly one repair attempt, then fail loudly into the trace. This becomes the handoff protocol later, so it is load-bearing.
- **B · The ledger (M3 ★★).** `errors`, `srs_cards`, `facts`, with CRUD the agent drives rather than the code. Ownership enforced in the data layer: only the owning agent may write its tables.
- **C · Own tools (T1 ★★★).** `dictionary`, `conjugate`, `kanji`, registered through the registry, plus the native-versus-prompted dual path decided by `/api/show` capabilities.
- **D · Metrics v1 (EV3 ★★).** Cost per turn, tokens, latency, TTFT, read straight off the logged rows. Ugly is fine; numbers are the point.

> **Done when:** you write a wrong Japanese sentence, and next session the assistant brings up that exact mistake.

LAYER 4 · Multi-agent — A2 ★★★, A1 ★★★, M2 ★★

The most valuable layer in the project. Everything here is new architecture, not features.

- **A · Orchestration (A2 ★★★).** Orchestrator, agent executor, typed handoff, and the guards: max hops, cycle detection, turn budget, per-agent timeout, degrade-to-sensei on any failure.
- **B · Per-agent context (M2 ★★).** The provider protocol (`priority`, `max_share`, `render`), the context builder, and the estimate-then-calibrate budget using `prompt_eval_count`.
- **C · The corrector (A1 ★★★).** Its spec file plus the ReAct strategy (A1): think, call a tool, verify, answer — on a context of roughly 800 tokens, not the whole conversation.
- **D · The trace view (EV3 ★★).** Per-agent-run logging and a collapsible per-turn trace. **This is your debugger for the rest of the project — build it now, not in week five.**

> **Done when:** one learner turn produces two agent runs, the trace shows both, and the guards visibly stop a deliberate infinite loop.

LAYER 5 · Knowledge and routing — R1 ★★, T1 ★★★, EV1 ★★★

- **A · The router (A2 ★★★).** Three-tier cascade (rules → embeddings → classifier), confidence floor falling back to sensei, explicit user override, every decision logged with its tier.
- **B · Compression (M2 ★★).** The sliding-window summarisation pass and the summary provider that consumes it.
- **C · RAG plus external MCP (R1 ★★, T1 ★★★).** Ingest, chunk with JLPT metadata, embed via `bge-m3`, brute-force cosine, metadata filter, citations — and wire in one pre-existing MCP server alongside your own tools.
- **D · The judge (EV1 ★★★).** Its spec, and claim-by-claim verification against the exact retrieved chunks, with unsupported claims flagged rather than silently passed.

> **Done when:** a grammar answer cites its source, and you can show the judge catching an unsupported claim live.

LAYER 6 · The three-star finish — R2 ★★★, EV2 ★★★, A5 ★, A4 ★

The catalogue rates R2 and EV2 at three stars each. An earlier version of this plan had both in the optional layer, which would have thrown away six stars. They belong here.

- **A · Persona and prompts (A5 ★, A4 ★).** Switch persona mid-session with history retained, and store prompts content-hashed so every run records its version.
- **B · The curriculum agent (M3 ★★).** Its spec and the SRS scheduling algorithm — and it runs *after* the visible response, never before.
- **C · Advanced RAG (R2 ★★★).** Reranking of retrieved chunks, per-chunk relevance scoring, and metadata filtering by JLPT level. A delta on R1, and rated higher than R1 — the cheapest three stars in the project.
- **D · Guardrails (EV2 ★★★).** Both interceptors: on input a scope check and PII detection before the model sees anything; on output redact or refuse. Cheap-first cascade — patterns, then embeddings, then a model call only when uncertain.

> **Done when:** a grammar answer survives reranking and cites a better source than R1 gave, and an out-of-scope or PII-bearing prompt is visibly refused.

LAYER 7 · Surface and proof — X1 ★, EV4 ★★, A6 ★★

- **A · Web UI backend (X1 ★).** FastAPI plus SSE endpoints over the existing event stream. No backend logic changes — it only subscribes.
- **B · A6 semantic cache (★★).** Cheap, because the embedding infrastructure already exists, and it pairs well with the router's tier 2. Optional.
- **C · Adversarial suite (EV4 ★★).** Jailbreaks, prompt injections, malformed inputs. Document where it breaks and where it holds. **This is the test suite that proves EV2 works, so the two are worth five stars as one block.**
- **D · Web UI frontend (X1 ★).** Streaming display, live context bar, trace view, agent switcher. **X1 is one star — keep it minimal and do not gold-plate it.** The trace view is the part that earns its keep, because it makes A2 visible.

LAYER 8 · Delivery — unrated, and the week everyone forgets to plan

- **A · The demo.** A written demo script and a seeded database, so you never depend on typing a full session live.
- **B · The number.** Measure tokens per turn, crew versus monolith. This is the slide that defends the whole architecture — and if it comes out badly, report it honestly.
- **C · Fallbacks.** Record every demo as video. The brief says a crash on stage is a preparation failure, not a technical one.
- **D · The repository.** README with setup and usage, `requirements.txt`, and a check that every implemented feature has its two user stories.

> **Done when:** you have run the whole keynote twice, on the machine you will present from.

What actually blocks what

- **Layer 1 → everything.** The four contracts are the only true bottleneck in the project.
- **Layer 2's bake-off → layers 3–6.** Every agent spec names a model. Settle it in layer 2.
- **Layer 3's structured output → layer 4's handoffs.** The handoff envelope *is* a validated JSON payload.
- **Layer 3's logging → layer 4's budget calibration and layer 6's context bar.** You cannot budget what you do not measure.

- **Layer 4's trace view → layers 5–7.** Debugging a crew without it costs more than building it.
- **Layer 5's RAG → layer 5's judge.** The judge checks claims against retrieved chunks, so C must land ingest before D can verify anything. This is the one **intra-layer dependency in the whole plan** — C should ship the retrieval interface early in the layer and let D work against fixtures.

If you fall behind — cut by rating, not by instinct

1. **A6 semantic cache (★★)** — the least integrated thing in the plan.
2. **The curriculum agent** — fold its scheduling logic into the corrector. You keep M3; you lose one crew member.
3. **EV4 (★★)** — only if EV2 itself is solid, since EV4's whole job is proving EV2.
4. **X1 Web UI (★)** — demo from the CLI instead. One star for the largest remaining block of work, so this is the cheapest big saving and the smallest loss of marks. **Keep the trace view even if you drop the browser** — build it into the CLI.

Never cut: T1, R2, A1, A2, EV1, EV2. Those are the six three-star features this plan is built to deliver, and they are the reason the architecture holds together. Losing one costs more than losing everything in the list above.